fastless

A less-compatible pager for very large and compressed files

User manual for version 1.0

October 2026

fastless pages files of hundreds of gigabytes with the keys, options, and prompts of GNU less. It maps files instead of reading them, decodes gzip, zstd, xz, and bzip2 on its own, and searches with every core.

Copyright © 2026 the fastless authors. fastless is free software, distributed under the GNU General Public License, version 3.

Contents

1 Introduction **1**

2 Getting started **1**
2.1 Building and installing . . . 1
2.2 A first look . . . 1

3 Moving around **2**
3.1 Scrolling . . . 2
3.2 Jumping . . . 2
3.3 Marks . . . 3

4 Searching **3**
4.1 Search commands . . . 3
4.2 Three search modes . . . 4
4.3 Case . . . 4
4.4 Pattern modifiers . . . 4
4.5 Filtering . . . 5
4.6 Searching without the pager . . . 5

5 Files, compressed input, and pipes **5**
5.1 Several files . . . 5
5.2 Compressed files . . . 5
5.3 Pipes and standard input . . . 6
5.4 Input preprocessors . . . 6

6 Display, line numbers, and prompts **6**
6.1 How text is shown . . . 6
6.2 Line numbers . . . 6
6.3 Prompts . . . 7

7 Other commands **7**
7.1 Editing a prompt . . . 7

8 Options **8**
8.1 Searching . . . 8
8.2 Display . . . 8
8.3 Quitting and the bell . . . 9
8.4 Input and resources . . . 9
8.5 Accepted, but without effect . . . 10
8.6 Not supported . . . 10

9 Key bindings and the environment **10**
9.1 lesskey files . . . 10
9.2 Environment variables . . . 11
9.3 Exit status . . . 11

10 Differences from GNU less **11**
10.1 What fastless adds . . . 11
10.2 What behaves differently . . . 12
10.3 What is not supported . . . 12

11 Performance and limits **13**
11.1 What to expect 13
11.2 Limits 13
11.3 Licence 13

1 Introduction

fastless is a terminal pager in the tradition of `more` and GNU `less`. If you know `less`, you already know how to use it: the keys, the command-line options, the `LESS` variable, prompts, marks, lesskey files, and input preprocessors all work the same way. What is different is the scale it is built for.

- **Huge files open instantly.** A plain file is mapped into memory read-only. Nothing is copied and no line index is built, so a 200 GB log opens as fast as a small one, and memory use does not grow with the file.
- **Searches use every core.** A search that runs further than 8 MiB is split into pieces that are searched in parallel; the first match in file order is still the one reported. Literal searches use AVX2 instructions on CPUs that have them.
- **Compressed files are understood natively.** gzip, zstd, xz, and bzip2 are recognised by their content, decoded (on several threads for xz and bzip2), and verified before the first page is shown.
- **Three ways to search.** Besides regular expressions, fastless can search for literal bytes, which is the fastest, or match whole lines against a glob pattern.
- **Pipes are paged as they arrive**, just as in `less`, without slowing down ordinary files.

Section 10 lists everything that behaves differently from `less`.

2 Getting started

2.1 Building and installing

fastless is written in C and builds with GCC or Clang on 64-bit Linux and macOS. It needs `pkg-config`, libzstd, liblzma, and a gzip decoder. Clone the repository with its submodules, which supply zlib-ng and the lbzip2 decoder:

```
git clone --recursive https://github.com/dvquilting/fastless.git
cd fastless
make
sudo make install          # into /usr/local; PREFIX=... to change
```

On Debian and Ubuntu, the build dependencies are:

```
sudo apt install build-essential pkg-config cmake libisal-dev zlib1g-dev \
                 libbz2-dev libzstd-dev liblzma-dev
```

For gzip, the build uses Intel ISA-L when it is installed, otherwise the bundled zlib-ng, otherwise the system zlib; `make ZLIB=isal`, `ZLIB=ng`, or `ZLIB=system` chooses one. bzip2 uses the bundled lbzip2 decoder, or the system libbz2 with `make BZIP2=system`. `make check` runs the test suites, and `make doc` rebuilds this manual.

2.2 A first look

```
fastless huge.log                  # page a file
fastless huge.log.zst other.log.gz # compressed files, two of them
some-command | fastless            # a pipe, shown as it arrives
LESS=-iR fastless build.log        # options from the environment
fastless +G --literal app.log      # start at the end, literal searches
```

Inside the pager, `Space` and `b` move a window forward and back, `j` and `k` a line, `g` and `G` go to the start and the end, and `/` searches. `h` shows a summary of every command, and `q` quits.

The status line at the bottom is fastless's own. It shows the file name, the byte position and percentage, the search mode, and a reminder of the main keys:

```
huge.log | byte 1048576/214748364800 (0.0%) | regex | q quit; h help; / search
```

`-m`, `-M`, and `-P` switch to `less`'s prompts instead (Section 6).

When standard output is not a terminal, fastless does not page at all: it copies its input to standard output, decoded if it was compressed. So `fastless huge.log.zst | grep ERROR` works like `zstdcat`.

3 Moving around

Commands are single keys, typed without `Return`. Those marked * may be preceded by a number N: `10j` moves ten lines, `50%` goes halfway, and `3n` finds the third match. `^X` means Control-X, and `ESC-x` means Escape followed by x.

3.1 Scrolling

Keys	Action
`Space` `f` `^F` `^V` `PgDn`*	Forward one window, or N lines.
`b` `^B` `ESC-v` `PgUp`*	Backward one window, or N lines.
`z` / `w`*	Forward / backward a window, setting the window size to N.
`ESC-Space`*	Forward a window, even at the end of the file.
`j` `e` `Return` `^E` `^N` `Down`*	Forward one line, or N lines.
`k` `y` `^Y` `^P` `^K` `Up`*	Backward one line, or N lines.
`J` / `K` `Y`*	Forward / backward a line, even past either end.
`d` `^D` / `u` `^U`*	Forward / backward half a window, setting the half-window to N.
`ESC-)` `Right` / `ESC-(` `Left`*	Right / left half a screen, or N columns.
`ESC-}` `^Right`	Right to the end of the longest line on the screen.
`ESC-{` `^Left`	Back to the first column.
`F`	Follow the end of the file, like `tail -f`. Any key stops.
`ESC-F`	Follow, stopping when the search pattern appears.
`r` `^R` `^L`	Repaint the screen.
`R`	Repaint, first taking in growth of the file.

Long lines are never wrapped; they end with a > in the last column, and the horizontal commands bring the rest into view.

3.2 Jumping

Keys	Action
`g` `<` `ESC-<` `Home`*	The first line, or line N.
`G` `>` `ESC->` `End`*	The last line, or line N.
`p` `%`*	N percent into the file; decimals such as `37.5%` work.
`P`*	Byte offset N. fastless
`{` `(` `[`*	The bracket closing the first such bracket on the top line.
`}` `)` `]`*	The bracket opening the last such bracket on the bottom line.
`ESC-^F` *a b*	The same for any pair of characters *a* and *b*; `ESC-^B` backward.

3.3 Marks

Keys	Action
`m`*x* / `M`*x*	Mark the top / bottom line with the letter *x*.
`'`*x* or `^X^X`*x*	Go to mark *x*.
`''`	Back to the previous position, before the last jump.
`'^` / `'$`	The start / end of the file.
`ESC-m`*x*	Clear mark *x*.

With `--save-marks`, marks are kept in the history file between sessions. With `--mouse`, the wheel scrolls (`--wheel-lines` at a time) and a click sets mark `#`.

4 Searching

4.1 Search commands

Keys	Action
`/`*pattern**	Search forward for the (N-th) line with a match.
`?`*pattern**	Search backward.
`n` / `N`*	Repeat the search in the same / opposite direction.
`ESC-n` / `ESC-N`*	Repeat, continuing into the following / preceding files.
`&`*pattern*	Show only the lines that match; `&` alone shows all lines again.
`ESC-u` / `ESC-U`	Toggle highlighting / forget the pattern.
`ESC-r` `ESC-l` `ESC-w`	Regular-expression, literal, or glob mode. fastless

A match is reported by its byte offset, and the line holding it is placed on the jump target line (`-j`; the top line by default). Every match on the screen is highlighted, overlapping ones included; `-g` highlights only the one found and `-G` none. A new forward search starts at the target line; `-a` starts it after the screen, and `-A` just after the target line. `--incsearch` searches as you type, each keystroke cancelling the search still running.

4.2 Three search modes

Mode	Matches
Regex `--regex`, `ESC-r` the default	POSIX extended regular expressions, such as `^ERROR.*timeout`, matched within each line without its newline.
Literal `--literal`, `ESC-l`	The pattern's exact bytes, anywhere, even across line ends. The fastest mode, using AVX2 where available.
Glob `--glob`, `ESC-w`	A pattern for the *whole line*: `*` matches any bytes, `?` one byte, and `\` quotes the next character. Use `*text*` to find *text* anywhere in a line. Bracket classes are not supported.

Switching modes recompiles the current pattern, so `n` goes on with the same text under the new rules. A regex or glob is not run on every line: fastless first works out the longest run of plain characters that every match must contain, finds the lines holding it with the literal matcher, and runs the full pattern only on those. For `^ERROR.*timeout`, only lines containing `ERROR` are examined. For alternatives such as `timeout|refused`, a run from each alternative is looked for, and only lines holding one are examined.

> **Tip.** For the fastest searches of very large files, use literal mode, and give a starting point with a mark, a percentage, or `P` so that less of the file has to be read from storage.

4.3 Case

`-i` ignores case when the pattern contains no capital letters ("smart case"), `-I` always ignores it, and `--case-sensitive` never does. Matching is done in bytes in the C locale: case folding covers ASCII letters only, and a UTF-8 pattern matches its encoded bytes.

4.4 Pattern modifiers

As in `less`, a pattern may begin with modifiers, which also change the prompt's label:

Modifier	Effect
`^N` or `!`	Find lines that do *not* match.
`^E` or `*`	Continue through the following (or preceding) files.
`^F` or `@`	Start at the first file (`/`) or the last (`?`).
`^K`	Highlight the matches without moving.
`^R`	Search literally, for this pattern only.
`^S` *n*	Match parenthesised subexpression *n* and highlight just it.
`^W`	Wrap to the other end of the file if nothing is found.
`^L`	Enter the next character literally.

The punctuation forms `!`, `*`, and `@` are modifiers only in regex mode, because glob patterns such as `*error*` and literal ones such as `!important` begin with them. `--search-options=EFKNRW` sets modifiers for every search. `n` and `N` keep the modifiers except `^F` and `^K`, and an empty pattern reuses the current one with new modifiers.

4.5 Filtering

`&`*pattern* shows only matching lines (with `^N`, only those that do not match), and movement, searches, and marks then refer to the lines shown. Filtering keeps no index: each line shown is found with the same threaded search, so a page of sparse matches costs a scan of the bytes between them.

4.6 Searching without the pager

`--find` prints the zero-based byte offset of the first match and exits, and with `--all`, every match, overlapping ones included. `--offset` starts the search at a byte position. Glob mode prints one offset per matching line.

```
fastless --find '^ ERROR.*timeout' huge.log
fastless --literal --find needle --all huge.log.zst
fastless --literal --find needle --offset 150000000000 huge.log
```

The exit status is 0 if something matched, 1 if nothing did, 2 for an error, and 130 if the search was interrupted.

5 Files, compressed input, and pipes

5.1 Several files

Keys	Action
`:e` [*file*] `^X^V`	Examine a new file; `:e` alone rereads the current one.
`:n` / `:p`*	The next / previous file named on the command line.
`:x`*	The first (or N-th) file.
`:d`	Remove the current file from the list.
`=` `^G` `:f`	Show the file name, position, and size.

Only the current file is kept open. Moving to another file releases a decoded file's memory, and coming back decodes it again. Standard input, pipes, and devices cannot be read twice, so they stay open with what was read.

5.2 Compressed files

gzip, zstd, xz, and bzip2 input is recognised by its first bytes, so names and extensions do not matter, and `--compression` can force a format or `none`. Concatenated gzip members, zstd frames, xz streams, and bzip2 streams are all read; only zero bytes may follow the last one. The whole file is decoded and its checksums verified before the first page appears. Progress is shown on standard error, and `^C` cancels. Damaged or truncated data is an error, never a partial file.

Format	Decoder
gzip	Intel ISA-L or zlib-ng; one thread.
zstd	libzstd; one thread, windows up to 128 MiB.
xz	liblzma; several threads when the file has several blocks, as threaded encoders write.

Format	Decoder
bzip2	lbzip2's block decoder on every thread, or libbz2 on one.

Decoded data is kept in memory up to `--max-mem` (10 GiB unless set; fractions such as `0.5` work). Larger output moves to a temporary file, created with mode 0600 in `--tmpdir`, `$TMPDIR`, or */tmp* and deleted at once, so it disappears when fastless exits for any reason.

> **Plan for the disk space.** Decoded data can be many times the size of the compressed file, and the temporary file needs room for all of it. On Linux, */tmp* is often held in memory: for large files, point `--tmpdir` at a disk.

5.3 Pipes and standard input

Without a file name, or with `-`, fastless reads standard input and takes keys from */dev/tty*. Named pipes open like files; devices need `-f`.

Streams are shown while they are still being written, so `make 2>&1 | fastless` and `tail -f log | fastless` show output as it arrives. While a stream is being read, its size is not known: the status shows *(reading)*, moving past the end waits for more data, `G` and `%` follow the end until the input ends, and forward searches go on into input as it arrives. Any key ends a wait.

Up to `--max-mem` (but at least 64 MiB) is read ahead so that the writer is never held up. Beyond that, input is read only as far as the screen, a search, or a jump needs, so `yes | fastless` does not fill the disk. A compressed stream is read whole, then decoded. When standard output is not a terminal, a pipe is copied straight through in constant memory.

5.4 Input preprocessors

`LESSOPEN` and `LESSCLOSE` work as in `less`: `|cmd %s` pages the command's output, `||cmd %s` also accepts empty output, and `cmd %s` names a replacement file. Files that begin with a compressed format's magic bytes skip the preprocessor, because the built-in decoders are faster; `-L --compression=none` shows their raw bytes. `-L` ignores `LESSOPEN` altogether, and `--show-preproc-errors` reports failures.

6 Display, line numbers, and prompts

6.1 How text is shown

Text is shown as UTF-8 when `LESSCHARSET` or the locale says UTF-8; otherwise bytes above 127 appear as `<XX>`. Control characters appear in bold as `^X`, and invalid or unprintable characters as `<XX>` or `<U+XXXX>`, so a file can never send escape sequences to the terminal. `-R` passes ANSI colour sequences through. As in `less`, an overstruck character is bold, an underscore overstrike underlines, a carriage return before a newline is hidden, and tabs expand to the `-x` stops.

6.2 Line numbers

`-N` shows a line number before each line. Prompts can show line numbers too: `-M` and `=` give *lines 6–12/100*. Nothing is counted unless something on the screen shows a number, so without `-N` or such a prompt there is no cost at all. When numbers are needed, newlines are counted with SIMD on all search threads and remembered per 64 MiB piece. A long jump into uncounted territory never waits: the screen appears at once, the numbers fill in when they are known, and

a counter shows the progress of longer counts. `^C` during counting turns line numbers off, as in `less`.

6.3 Prompts

`-m`, `-M`, and `-P` select `less`'s medium, long, or custom prompts and its prompt language. `-Ps` (or `-P` alone) sets the short prompt, `-Pm` and `-PM` the medium and long ones, `-P=` the `=` message, `-Ph` the help prompt, and `-Pw` the prompt while waiting for a pipe. The escapes are:

`%f`	file name	`%l, %L`	line number, last line
`%F`	last component of the name	`%d, %D`	page number, page count
`%g`	name, quoted for the shell	`%c`	column of the left edge
`%i, %m`	file index, number of files	`%s`	size, as %B
`%x`	next file's name	`%t, %T`	trim spaces / the word "file"
`%b, %B`	byte offset, size	`%E`	the editor
`%p, %P`	percent, by bytes / by lines	`?`*x*`...:...`	conditionals

A line-number item shows `?`, and its conditional is false, until the number is known. While a pipe is still being read, the conditions on its size (`?s`, `?B`, `?p`, `?P`, `?L`, `?D`, `?e`) are false.

7 Other commands

Keys	Action
`-`*x* / `--`*name*	Toggle or set an option; `-+`*x* resets it, `-!`*x* sets its opposite.
`_`*x* / `__`*name*	Show an option's setting.
`+`*cmd*	Run *cmd* in each file examined from now on.
`!`*command*	Run a shell command: `%` is the file name, `#` the previous file, `!!` the last command.
`#`*command*	The same, expanding prompt escapes.
`|`*x command*	Pipe the text from mark *x* (or `^`, `$`, `.`) through the screen into *command*.
`s` *file*	Save the input; for compressed files, the decoded data.
`v`	Edit the file with `$VISUAL`, `$EDITOR`, or `vi`, at the current line.
`h` `H` / `V`	Help / version.
`q` `Q` `:q` `:Q` `ZZ`	Quit, or leave the help.
`^C`	Cancel a search, count, wait, or `F`.
`^Z`	Suspend.

7.1 Editing a prompt

Prompts for patterns, file names, and commands have `less`'s line editing: `Left` `Right` (`ESC-h` `ESC-l`), word moves with `^Left` `^Right` (`ESC-b` `ESC-w`), `Home` `End` (`ESC-0` `ESC-$`), `Del` (`ESC-x`), word deletion with `ESC-Bksp` and `ESC-X`, `^U` to clear the line, `^V` to enter a character literally, and `Up` `Down` for history, matching what was typed. `Bksp` on an empty prompt, `^G`, or a lone `ESC` cancels. History is kept in *~/.fastlesshst* (`FASTLESSHISTFILE`).

8 Options

Options are read from the `LESS` environment variable first, then from the command line. Long options may be shortened to any unique prefix, and most can be changed inside the pager with `-`. In `LESS`, string values end at `$` and unknown options are ignored. Options marked fastless are not in `less`.

8.1 Searching

Option	Meaning	
`--regex`, `--literal`, `--glob`	The search mode.	fastless
`-i`, `--ignore-case`	Ignore case if the pattern has no capitals.	
`-I`, `--IGNORE-CASE`	Always ignore case.	
`--case-sensitive`	Turn `-i` and `-I` off.	fastless
`-a`, `--search-skip-screen`	Start forward searches after the screen.	
`-A`, `--SEARCH-SKIP-SCREEN`	Start them just after the target line.	
`-g`, `--hilite-search`	Highlight only the match found.	
`-G`, `--HILITE-SEARCH`	Highlight nothing.	
`--incsearch`	Search while the pattern is typed.	
`--search-options=`*EFKNRW*	Modifiers for every search; - clears those before it.	
`-p`*pattern*, `--pattern`	Search when the first file opens.	
`-j`*n*, `--jump-target`	The target line: a line (negative from the bottom) or a fraction such as `.5`.	
`--match-shift=`*n*	Columns kept left of a match scrolled into view.	
`--find=`*pattern*	Print the first match's byte offset and exit.	fastless
`--all`	With `--find`, print every match.	fastless
`--offset=`*byte*	Start the pager or the search at a byte offset.	fastless

8.2 Display

Option	Meaning
`-R`, `--RAW-CONTROL-CHARS`	Pass ANSI colour sequences through.
`-r`, `--raw-control-chars`	The same as `-R`.
`-N`, `--LINE-NUMBERS`	Show line numbers.
`-n`, `--line-numbers`	Show no line numbers, in prompts either.
`--line-num-width=`*n*	Width of the number column (7).
`--no-number-headers`	Leave header lines unnumbered.
`-s`, `--squeeze-blank-lines`	Show runs of blank lines as one.
`-x`*n,...*, `--tabs`	Tab stops (8).
`-~`, `--tilde`	No ~ after the end of the file.
`-#`*n*, `--shift`	Horizontal step: columns, a fraction, or 0 for half the screen.

Option	Meaning
`--header=`*L,C,N*	Keep *L* lines (from line *N*) and *C* columns in place.
`--no-search-header-lines`	Searches skip header lines; `--no-search-header-columns` columns, `--no-search-headers` both.
`-J`, `--status-column`	A column marking marks and matching lines.
`--status-col-width=`*n*	Its width (2).
`--status-line`	Highlight marked lines.
`-u`, `--underline-special`	Show backspaces and carriage returns as control characters.
`-U`, `--UNDERLINE-SPECIAL`	Also tabs.
`--proc-backspace`, `--proc-return`, `--proc-tab`	Process that character whatever `-u` says.
`--PROC-BACKSPACE`, `--PROC-RETURN`, `--PROC-TAB`	Show it as a control character whatever `-u` says.
`--rscroll=`*c*	The mark for a chopped line (>; - for none).
`-w`, `--hilite-unread`	Highlight the first new line after a page forward.
`-W`, `--HILITE-UNREAD`	... after any forward movement.
`-m`, `--long-prompt`	`less`'s medium prompt.
`-M`, `--LONG-PROMPT`	`less`'s long prompt.
`-P`*prompt*, `--prompt`	A custom prompt (Section 6).
`-c`, `--clear-screen`, `-C`, `--CLEAR-SCREEN`	Repaint by clearing the screen.
`-X`, `--no-init`	Use the normal screen, not the alternate one.
`--redraw-on-quit`	Leave the last screen on the terminal.
`-z` *n*, `--window`	Window size; negative counts from the screen height.
`--mouse`, `--MOUSE`	Wheel scrolling; upper case reverses it.
`--wheel-lines=`*n*	Lines per wheel step (1).

8.3 Quitting and the bell

Option	Meaning
`-e`, `--quit-at-eof`	Quit on a forward command at the end.
`-E`, `--QUIT-AT-EOF`	Quit as soon as the end is shown.
`-F`, `--quit-if-one-screen`	Print a file that fits on one screen and exit.
`-K`, `--quit-on-intr`	Quit with status 2 on `^C`.
`--intr=`*c*	Another character that acts as `^C` (`^X`).
`-q`, `--quiet`, `--silent`	Ring the bell only for errors.
`-Q`, `--QUIET`, `--SILENT`	Never ring the bell.

8.4 Input and resources

Option	Meaning	
`--compression=`*format*	`auto`, `none`, `gzip`, `zstd`, `xz`, or `bzip2`.	fastless
`--max-mem=`*GiB*	Memory for decoded or piped input before a temporary file (10).	fastless
`--tmpdir=`*dir*	Where temporary files go.	fastless
`--threads=`*n*	Threads for searching, counting, xz and bzip2: 1–256, or 0 for one per CPU up to 32.	fastless
`--simd=`*mode*	`auto`, `off`, or `avx2` for literal searches.	fastless
`-f`, `--force`	Open devices and other non-regular files.	
`-L`, `--no-lessopen`	Ignore `LESSOPEN`.	
`--show-preproc-errors`	Report a failing preprocessor.	
`--follow-name`	`F` reopens a file replaced under its name.	
`--lesskey-src=`*file*	Key bindings from a lesskey source file.	
`--lesskey-content=`*text*	Key bindings as text; lines end at newlines or `;`.	
`--save-marks`	Keep marks between sessions.	
`--no-histdups`	Drop earlier copies of a command from the history.	
`+`*cmd*, `++`*cmd*	Run *cmd* in the first / every file, as in `+G` or `+/error`.	
`-V`, `--version`	Print the version.	
`-?`, `--help`	Print an option summary.	

8.5 Accepted, but without effect

`-b` (`--buffers`), `-B` (`--auto-buffers`), `-d` (`--dumb`), `-h` (`--max-back-scroll`), `-S` (`--chop-long-lines`), `-y` (`--max-forw-scroll`), `--file-size`, `--no-keypad`, `--no-vbell`, and `--exit-follow-on-close` are accepted so that existing `LESS` settings keep working. Lines are always chopped, as with `-S`.

8.6 Not supported

`-o`, `--log-file`, `-O`, and `--LOG-FILE` (log files); `-t`, `--tag`, `-T`, and `--tag-file` (tags); `-D`, `--color`, and `--use-color` (colours); `--wordwrap`; `--modelines`; `-"` and `--quotes`; `--use-backslash`; and `-k` and `--lesskey-file` (compiled lesskey files) are refused with a message.

9 Key bindings and the environment

9.1 lesskey files

Keys can be rebound with `lesskey` source: `--lesskey-src`, or else the first of `$LESSKEYIN`, *$XDG_CONFIG_HOME/lesskey*, *~/.config/lesskey*, and *~/.lesskey*; then `LESSKEY_CONTENT` and `--lesskey-content`. The `#command`, `#line-edit`, and `#env` sections, `#stop`, `\k` key names, and extra strings work as in `lesskey`(1). Besides `less`'s actions there are `regex-mode`, `literal-mode`, and `glob-mode`; the tag and OSC 8 hyperlink actions are accepted but do nothing.

```
#command
t     forw-screen
x     literal-mode
#env
LESS = -iR
```

9.2 Environment variables

Variable	Use
`LESS`	Options read before the command line.
`LESSOPEN`, `LESSCLOSE`	Input preprocessors.
`LESSEDIT`	The editor command for `v`, in prompt language (`%E ?lm+%lm. %g`).
`VISUAL`, `EDITOR`	The editor; empty counts as unset (`vi`).
`SHELL`	The shell for `!`, `#`, and `|`.
`LESSSECURE`	When `1`: no shell commands, pipes, saving, editing, `:e`, preprocessors, or history.
`LESSCHARSET`	`utf-8` for UTF-8, anything else for single bytes; unset, the locale decides.
`LESSKEYIN`, `LESSKEY_CONTENT`	Key bindings.
`FASTLESSHISTFILE`	The history file (*~/.fastlesshst*; `-` for none).
`TMPDIR`	Temporary files, without `--tmpdir`.

9.3 Exit status

Status	Meaning
0	Success; for `--find`, something matched.
1	`--find` found nothing.
2	An error, such as an unreadable file or damaged compressed data, or `^C` with `-K`.
130	Interrupted by `^C` outside the pager, as while decoding.
128+n	Ended by signal n, such as SIGTERM.

10 Differences from GNU less

fastless follows `less` 668 wherever it can, so muscle memory, `LESS` settings, and lesskey files carry over. These are the places where it does not.

10.1 What fastless adds

Feature	Details
Search modes	Literal and whole-line glob searches beside regular expressions (`--literal`, `--glob`, `ESC-r` `ESC-l` `ESC-w`).
Built-in decoding	gzip, zstd, xz, and bzip2, detected by content, decoded on several threads, verified; `--compression`, `--max-mem`, `--tmpdir`.
Parallel search	Long searches and line counts use every core (`--threads`); literal matching uses AVX2 (`--simd`).
Byte positions	`P` jumps to a byte offset, and `--offset` starts there.
Batch search	`--find` and `--all` print match offsets without the pager.
Status line	A default status line with file, position, and search mode.

10.2 What behaves differently

Area	In fastless
Long lines	Always chopped, never wrapped: `-S` is the only behaviour, and `--wordwrap` is not supported.
Line numbers	Counted lazily, only when something shows them; a long jump does not wait for the count. `less` counts as it reads.
Regular expressions	POSIX extended expressions from the C library in the C locale, so case folding is ASCII only. Literal searches also match across line ends.
Positions	Offsets, percentages, and the status line count bytes of the decoded data.
Preprocessors	Compressed files bypass `LESSOPEN`. Standard input is never preprocessed, even with the `|-` form.
Control characters	`-r` is the same as `-R`: only colour sequences reach the terminal, and `-u` never sends raw backspaces or carriage returns. Other bytes show as `^X`, `<XX>`, or `<U+XXXX>`; `LESSBINFMT` is not used.
`-?`	Prints an option summary and exits, rather than opening the help screen.
History	Kept in *~/.fastlesshst* (`FASTLESSHISTFILE`), apart from `less`'s.

10.3 What is not supported

- Tags (`-t`, `-T`, `:t`), log files (`-o`, `-O`), colours (`-D`, `--use-color`), `--modelines`, `-"`, `--use-backslash`, OSC 8 hyperlinks, and compiled lesskey files (`-k`).
- The *-n* form of *-zn* (as in `-20`), and capitalising only the first letter of an upper-case long option (as in `--Quit-at-eof`).
- Environment variables not listed above, such as `LESSHISTFILE`, `LESSANSIENDCHARS`, `LESSMETACHARS`, `LESS_IS_MORE`, and the `LESS_TERMCAP_*` variables.
- Character sets other than UTF-8 and single bytes.

11 Performance and limits

11.1 What to expect

Opening a plain file costs nothing, whatever its size. Searching reads the bytes it covers from storage, so a cold search runs at disk speed; once the file is cached, literal searches run at several GiB per second. On an 8-core Ryzen 7 5700X with a warm cache, a case-insensitive literal search measured about 8.2 GiB/s, and a regex without a usable literal ran 6.7 times faster on eight threads than on one. Numbering the end of a 4 GiB, 84-million-line file took 0.15 s.

bzip2 decoding, 556 MiB of logs compressed by `bzip2 -9`, wall time:

Threads	bzip2 (libbz2)	lbzip2	fastless
1	11.6 s	9.5 s	9.4 s
4	—	2.4 s	2.4 s
16	—	1.8 s	1.8 s

11.2 Limits

- A mapped file must not be truncated while it is shown, or the program may be killed by SIGBUS, as with any program that maps files. Replacing the file by name changes nothing unless `F` is used with `--follow-name`.
- A regular expression cannot match within a single line longer than 2 GiB; literal and glob searches can.
- Archive members, such as those of a tar file, are not extracted; only the compression is removed.
- `^C` is checked between pieces of work, so one very long line or one costly regular-expression call can delay it.
- fastless needs a 64-bit system with POSIX `REG_STARTEND`, such as Linux or macOS.

11.3 Licence

fastless is free software under the GNU General Public License, version 3. It includes the bzip2 block decoder of lbzip2, which is licensed under the GPL, version 3 or later.